

Universidade Federal de Santa Maria
Curso de Sistemas de Informação
Disciplina de Qualidade de Software – 2026/02

Relatório de testes automatizados do Truco Gaudério

Lucas Salvini Bertol e Renato Marquioro
Professor Luis Alvaro

Santa Maria, RS – novembro de 2026

1 Introdução

O trabalho avaliou o jogo Truco Gaudério, implementado em Python. O jogo permite uma partida entre uma pessoa e um bot que usa raciocínio baseado em casos (CBR). O relatório apresenta os requisitos do jogo, os cenários de uso, os testes implementados e os resultados obtidos com a suíte de testes.

2 Metodologia

A suíte usa Pytest. Os testes unitários verificam regras, retornos de funções e respostas a diferentes parâmetros. Os testes de cenário executam o fluxo do jogo com mãos fixas e entradas simuladas. Para avaliar a IA, há testes que consultam a base real de casos e testes que controlam a resposta do bot para isolar o comportamento do jogo.

A suíte cobre as cinco categorias obrigatórias do enunciado. A tabela identifica exemplos de cada categoria.

Categoria	Exemplos na suíte de testes
Requisitos e cenários de uso	CT42 e CT43 verificam a vitória em uma mão e o fim da partida.
Chamadas com parâmetros variados	CT04 e CT10 comparam cartas e calculam Envido com entradas variadas.
Entradas do usuário	CT11, CT22 e CT32 simulam respostas válidas e inválidas.
Retornos de funções	CT08, CT18 e CT37 verificam o vencedor, a aceitação do Truco e a resposta da IA.
Inteligência artificial	CT35 a CT40 consultam a base e exercitam as decisões da CBR.

3 Requisitos do jogo

Os requisitos foram levantados a partir do README, do código, da planilha de codificação e dos comentários do autor. O limite de tempo de resposta da IA é um critério definido pelo grupo.

ID	Requisito funcional
RF01	Usar um baralho espanhol de 40 cartas, com os números de 1 a 7 e de 10 a 12 em cada um dos quatro naipes.
RF02	Distribuir automaticamente três cartas aleatórias a cada jogador.
RF03	Ordenar as cartas comuns por força (4, 5, 6, 7, 10, 11, 12, 1, 2 e 3). As manilhas, da menor para a maior, são 7 de Ouros, 7 de Espadas, 1 de Bastos e 1 de Espadas.
RF04	Dar a vitória da rodada à carta mais forte. Cartas de mesma força empatam.
RF05	Disputar cada mão em melhor de três rodadas. Duas vitórias encerram a mão, que vale um ponto sem Truco.
RF06	Dar ao vencedor da rodada o primeiro turno da rodada seguinte.
RF07	Atribuir 2 pontos ao Truco aceito, 3 ao Retruco e 4 ao Vale 4. Na recusa, quem pediu recebe o valor anterior. O mesmo jogador não pode aumentar duas vezes seguidas.
RF08	Ao ir ao baralho, encerrar a mão e conceder ao adversário o valor atual da aposta.
RF09	Permitir Envido na primeira rodada, com três cartas. Envido vale 2, Real Envido vale 5 e Falta Envido vale 12 menos os pontos do adversário. Na recusa, o autor do pedido recebe 1, 2 ou 5 pontos, respectivamente.
RF10	Calcular o Envido somando 20 a duas cartas do mesmo naipe. Sem par, usar a maior carta. As figuras 10, 11 e 12 valem zero nesse cálculo.
RF11	Dar o Envido a quem tiver mais pontos. A regra do sistema dá o empate ao jogador 1.
RF12	Permitir Flor com três cartas do mesmo naipe. Flor vale 3 e bloqueia o Envido. Se ambos têm Flor, há Contraflor, que vale 6, ou Contraflor e o Resto.
RF13	Encerrar a partida quando um jogador alcançar 12 pontos.
RF14	Exibir as cartas e as opções válidas de Truco, Flor, Envido, Real Envido, Falta Envido e desistência, conforme a mão e a rodada.
RF15	Avisar quando a entrada for inválida e pedir uma nova opção, mantendo a partida em andamento.
RF16	Usar CBR para decidir qual carta jogar e como responder a Truco, Envido e Flor com base nos casos semelhantes.
RF17	Consultar os 100 vizinhos mais próximos na base de casos `dbtrucoimitacao_maos.csv`.
RF18	Registrar cada mão no arquivo `jogadas.csv`.

ID	Requisito não funcional
RNF01	Iniciar o jogo por `python -m truco` com as dependências instaladas.
RNF02	Responder a uma consulta da IA em menos de um segundo, conforme o critério do grupo.
RNF03	Reiniciar cartas, rodadas e estados das apostas ao começar uma nova mão.

4 Cenários de uso e rastreabilidade

O fluxo principal e as alternativas descrevem o comportamento esperado. A última coluna identifica o que a suíte de testes exercita em cada cenário.

Cenário	Fluxo principal e alternativas	Testes atuais
UC01 – Iniciar partida	Informar os nomes e receber três cartas. Nomes vazios também podem ser informados.	CT34 e CT44 cobrem nomes e inicialização. CT02 cobre a distribuição.
UC02 – Jogar uma mão	Jogar cartas, resolver as rodadas e atualizar o placar. O bot também pode iniciar uma rodada.	CT42 executa uma mão com vitória humana e verifica a nova mão e o registro. CT04, CT08 e CT09 exercitam as regras de rodada.
UC03 – Humano pede Truco	O humano escolhe [4] e o bot aceita, recusa ou aumenta a aposta.	CT18 a CT21 e D04 exercitam as regras no controlador.
UC04 – Bot pede Truco	O bot pede Truco. O humano pode recusar, aceitar, aumentar ou digitar uma opção inválida.	CT19, CT20 e CT22 exercitam as respostas no controlador.
UC05 – Envido	Um jogador pede Envido, Real Envido ou Falta Envido. O outro responde e o vencedor recebe os pontos.	CT10 a CT16 cobrem as regras. UC05 executa a partida com pedido do bot no início. D03 e D07 exercitam defeitos em fluxos do jogo.
UC06 – Flor	Um jogador com três cartas do mesmo naipe canta Flor. Se ambos têm Flor, há Contraflor.	CT25 a CT27 exercitam a identificação, a pontuação e as respostas em testes unitários.
UC07 – Ir ao baralho	O humano escolhe [9], o bot ganha a aposta e uma nova mão começa.	CT33 executa o fluxo pelo menu principal.
UC08 – Entrada inválida	O humano digita texto ou uma opção inválida. O jogo deve avisar e repetir a pergunta.	CT32 verifica opções numéricas inválidas. D06 e D11 registram falhas no menu.
UC09 – Fim da partida	Ao chegar a 12 pontos, o jogo anuncia o vencedor e encerra.	CT43 verifica o placar final, o anúncio e o encerramento.

5 Execução

Na raiz do projeto, com o ambiente virtual ativo, instalam-se as dependências e executa-se a suíte com o comando abaixo. Os marcadores `ia`, `entrada` e `cenario` permitem executar grupos de testes.

```
python -m pip install -r requirements-test.txt
python -m pytest --cov=truco --cov-report=term -q
```

6 Resultados

A suíte de testes reuniu 60 testes em 12 arquivos. Foram registrados **50 testes aprovados, 10 falhas esperadas e nenhuma falha inesperada**. A cobertura de linhas do pacote `truco` foi de **79%** (868 de 1.095 linhas executáveis).

Arquivo	Aprovados	XFAIL
`test_00_ambiente.py`	3	0
`test_baralho_carta.py`	5	1
`test_cbr.py`	8	3
`test_cenarios.py`	3	1
`test_dados.py`	2	0
`test_entradas_usuario.py`	3	2
`test_envido.py`	7	1
`test_flor.py`	3	0
`test_interface.py`	4	0
`test_jogador_bot.py`	4	0
`test_jogo.py`	3	1
`test_truco.py`	5	1
Total	**50**	**10**

7 Defeitos verificados

A marcação `XFAIL` identifica testes que reproduzem defeitos conhecidos. A suíte manteve essas verificações separadas das que passam.

ID	Comportamento observado e situação
D01	O separador incorreto impedia o carregamento da base. Corrigido e verificado pelo CT44.
D02	Cartas de força igual dão a vitória ao jogador 1 em vez de empatar.
D03	O pedido de Envido no turno do bot gera um erro de argumentos. A chamada seguinte no código ainda atribui o pedido ao humano.
D04	O Truco é aceito, mas a mão continua valendo um ponto.
D05	O bot falha ao classificar uma mão com três cartas de mesma força.
D06	Texto na seleção de carta encerra o jogo com `ValueError`.
D07	A pergunta de aceitação do Envido dirigido ao humano cita o jogador 2.
D08	A IA lança `IndexError` ao buscar uma carta em um cenário com duas manilhas e nenhum vizinho vencedor.
D09	A IA aumenta o Truco em um cenário de mão fraca.
D10	O registro preenchido durante a partida não chega à instância de dados consultada pela IA.
D11	Um índice igual ao tamanho da mão é aceito pela validação e provoca `IndexError`.

8 Conclusão

A suíte de testes mantém verificações das regras do jogo, das entradas do usuário, dos retornos de funções e da IA. Os cenários automatizados demonstram o fluxo da partida, incluindo a pontuação da mão, a desistência, a disputa de Envido e o encerramento aos 12 pontos. Os resultados indicam quais comportamentos foram validados e quais ainda apresentam defeitos conhecidos.

Fontes

TRUCO GAUDÉRIO. README, código-fonte, planilha de codificação e comentários do autor. Materiais fornecidos para a atividade.

Inventário dos testes

A lista apresenta os testes por arquivo, com o comportamento verificado e o resultado observado.

`test_00_ambiente.py`

`test_todos_os_modulos_do_jogo_importam`

Importa os módulos do jogo. *(Aprovado).*

`test_CT44_base_de_casos_carrega_sem_adaptador`

Confere se a base carrega sem o adaptador dos testes. *(Aprovado).*

`test_CT44_python_m_truco_inicia`

Executa python -m truco e confere a inicialização. *(Aprovado).*

`test_baralho_carta.py`

`test_CT01_baralho_tem_40_cartas_10_por_naipe_sem_repeticao`

Confere 40 cartas, dez por naipe e nenhuma repetida. *(Aprovado).*

`test_CT01_limites_do_laco_que_cria_o_baralho`

Verifica quais números entram no baralho nos limites do laço. *(Aprovado).*

`test_CT02_CT03_distribuicao_e_embaralhamento`

Verifica a distribuição das cartas e a ordem após embaralhar. *(Aprovado).*

`test_CT05_pontos_e_ordem_de_forca_das_40_cartas`

Confere os pontos e a hierarquia das 40 cartas. *(Aprovado).*

`test_CT06_classificar_mao_em_alta_media_e_baixa`

Confere a classificação das cartas em alta, média e baixa. *(Aprovado).*

`test_D05_bot_recebe_tres_cartas_de_mesma_forca`

Verifica se o bot consegue receber três cartas de força igual. *(XFAIL).*

test_cbr.py

`test_CT35_base_de_casos_e_consulta_de_100_vizinhos`

Valida a base de casos e a consulta dos 100 vizinhos. *(Aprovado).*

`test_CT35_caso_da_base_recupera_ele_mesmo`

Confere se um caso da base recupera a si próprio com distância zero. *(Aprovado).*

`test_CT36_escolhe_sempre_uma_carta_da_mao`

Verifica se a IA retorna um índice válido da mão. *(Aprovado).*

`test_CT36_joga_a_carta_mais_proxima_do_valor_de_referencia`

Confere se a IA escolhe a carta mais próxima do valor de referência. *(Aprovado).*

`test_D08_ia_escolhe_carta_com_mao_de_duas_manilhas`

Verifica se a IA consegue escolher uma carta com duas manilhas na mão. *(XFAIL).*

`test_CT37_resposta_ao_truco_valida_e_crescente`

Confere os retornos válidos do Truco e sua progressão conforme a qualidade da mão. *(Aprovado).*

`test_D09_mao_fraca_nao_aumenta_o_truco`

Verifica se a IA evita aumentar o Truco com uma mão fraca. *(XFAIL).*

`test_CT38_envido_respostas_validas_e_pedido_so_com_pontos`

Confere as respostas ao Envido e quando a pontuação leva a um pedido. *(Aprovado).*

`test_CT39_mesma_situacao_mesma_decisao_em_menos_de_1_segundo`

Repete as decisões para a mesma situação e verifica o limite de um segundo. *(Aprovado).*

`test_CT40_situacao_informada_muda_os_vizinhos`

Verifica se mudar a situação altera os vizinhos recuperados. *(Aprovado).*

`test_D10_ia_recebe_a_situacao_da_partida`

Verifica se os dados registrados na partida chegam à IA. *(XFAIL).*

test_cenarios.py

`test_CT42_mao_completa_vencida_pelo_humano`

Simula a vitória do humano em uma mão e confere o placar, a nova mão e o CSV. *(Aprovado).*

`test_CT43_partida_termina_aos_12_pontos`

Confere se a partida termina quando o bot alcança 12 pontos. *(Aprovado).*

`test_UC05_bot_pede_envido_no_inicio_e_vence`

Simula o bot pedindo Envido no início da partida e vencendo a aposta. *(Aprovado).*

`test_D07_pergunta_do_envido_e_feita_ao_jogador_1`

Confere se a pergunta de aceitação do Envido é dirigida ao humano. *(XFAIL).*

test_dados.py

`test_CT41_primeira_rodada_grava_a_mao_do_bot`

Confere o registro da mão do bot na primeira rodada. *(Aprovado).*

`test_CT41_cada_mao_vira_uma_linha_em_jogadas_csv`

Verifica se cada mão acrescenta uma linha ao arquivo jogadas.csv. *(Aprovado).*

test_entradas_usuario.py

`test_D06_texto_na_escolha_da_carta_pede_de_novo`

Digita texto ao escolher a carta e espera uma nova tentativa. *(XFAIL).*

`test_CT32_opcao_inexistente_avisa_e_pergunta_de_novo`

Confere se uma opção inválida gera aviso e repete a pergunta. *(Aprovado).*

`test_D11_indice_igual_ao_numero_de_cartas`

Verifica se o jogo rejeita o índice igual ao número de cartas. *(XFAIL).*

`test_CT33_ir_ao_baralho_da_o_ponto_ao_bot_e_comeca_nova_mao`

Confere o ponto do bot e o início de uma nova mão após a desistência. *(Aprovado).*

`test_CT34_nomes_dos_jogadores`

Verifica se o jogo recebe e exibe os nomes dos jogadores. *(Aprovado).*

test_envido.py

`test_CT10_calculo_dos_pontos_de_envido`

Confere o cálculo dos pontos de Envido para diferentes mãos. *(Aprovado).*

`test_CT11_humano_recusa_ou_aceita_o_envido_do_bot`

Simula a recusa e a aceitação do pedido de Envido do bot. *(Aprovado).*

`test_CT11_opcao_invalida_repete_a_pergunta`

Verifica se uma opção inválida faz a pergunta se repetir. *(Aprovado).*

`test_CT12_humano_pede_envido_e_o_bot_responde`

Simula o pedido de Envido do humano e confere a resposta do bot. *(Aprovado).*

`test_CT13_CT14_valores_do_real_envido_e_da_falta_envido`

Confere a pontuação de Real Envido e Falta Envido. *(Aprovado).*

`test_CT15_vencedor_do_envido_inclusive_no_empate`

Confere o vencedor do Envido com pontos diferentes ou iguais. *(Aprovado).*

`test_CT16_segundo_pedido_na_mesma_mao_e_ignorado`

Verifica se um segundo pedido de Envido na mesma mão é ignorado. *(Aprovado).*

`test_D03_bot_pede_envido_na_sua_vez`

Força um pedido de Envido no turno do bot e verifica o processamento. *(XFAIL).*

test_flor.py

`test_CT25_flor_exige_tres_cartas_do_mesmo_naipe`

Confere se Flor exige três cartas do mesmo naipe. *(Aprovado).*

`test_CT26_flor_vale_3_e_contraflor_vale_6`

Confere três pontos para Flor e seis para Contraflor. *(Aprovado).*

`test_CT27_resposta_a_contraflor_com_opcao_invalida`

Confere se a pergunta de Contraflor se repete após uma resposta inválida. *(Aprovado).*

test_interface.py

`test_CT28_opcoes_mudam_conforme_a_mao`

Confere as opções exibidas conforme as cartas na mão. *(Aprovado).*

`test_CT29_moldura_das_mensagens`

Verifica a moldura das mensagens. *(Aprovado).*

`test_CT29_placar_e_vencedores`

Confere as mensagens de placar e de vencedores. *(Aprovado).*

`test_CT30_desenho_das_cartas`

Confere o desenho das cartas e os símbolos dos naipes. *(Aprovado).*

test_jogador_bot.py

`test_jogador_joga_cartas_e_acumula_pontos`

Confere a retirada de cartas da mão e a soma de pontos do jogador. *(Aprovado).*

`test_bot_consulta_a_ia_para_jogar_e_atualiza_a_mao`

Verifica se o bot consulta a IA e atualiza a mão após jogar. *(Aprovado).*

`test_bot_com_duas_cartas_pede_truco_conforme_a_ia`

Confere o pedido de Truco do bot com duas cartas conforme a resposta da IA. *(Aprovado).*

`test_CT24_nova_mao_zera_o_estado_e_mantem_os_pontos`

Confere a limpeza do estado da mão e a preservação dos pontos. *(Aprovado).*

test_jogo.py

`test_CT04_carta_mais_forte_vence_nos_1560_confrontos`

Compara os pares de cartas para verificar se a de maior força vence. *(Aprovado).*

`test_D02_cartas_de_mesma_forca_empatam`

Verifica se cartas de força igual resultam em empate. *(XFAIL).*

`test_CT08_adicionar_rodada_retorna_o_vencedor`

Confere o retorno do vencedor ao registrar uma rodada. *(Aprovado).*

`test_CT09_vencedor_da_rodada_joga_primeiro`

Verifica se o vencedor da rodada joga primeiro. *(Aprovado).*

`test_truco.py`

`test_CT18_humano_pede_truco_e_o_bot_aceita`

Confere o retorno de aceitação quando o humano pede Truco e o bot aceita. *(Aprovado).*

`test_D04_truco_aceito_vale_2_pontos`

Verifica se o Truco aceito aumenta o valor da mão para dois pontos. *(XFAIL).*

`test_CT19_retruco_vale_3_e_vale_4_vale_4`

Confere os valores de três pontos para Retruco e quatro para Vale 4. *(Aprovado).*

`test_CT20_recusa_entrega_o_valor_anterior_a_quem_pediu`

Verifica se a recusa concede a quem pediu o valor anterior da aposta. *(Aprovado).*

`test_CT21_mesmo_jogador_nao_aumenta_duas_vezes_seguidas`

Confere o bloqueio de dois aumentos seguidos pelo mesmo jogador. *(Aprovado).*

`test_CT22_humano_responde_truco_do_bot_com_opcao_invalida`

Verifica se uma resposta inválida ao Truco faz a pergunta se repetir. *(Aprovado).*